DIGITAL LOGIC DESIGN

LAB MANUAL

Using Logisim / Logisim Evolution

A Complete Practical Guide for Undergraduate Students

Combinational Logic • Sequential Circuits • Memory & PLDs

Ali Ahmad Aminu, PhD
Computer Science Department
Gombe State University

Preface

This laboratory manual has been carefully designed to provide undergraduate students of Computer Science and Computer Engineering with a comprehensive, hands-on foundation in Digital Logic Design. It bridges theoretical knowledge with practical circuit implementation using Logisim Evolution — a powerful, free, open-source digital logic simulator.

The manual progresses logically from basic gate-level design through combinational circuits, sequential systems, and finally programmable logic devices. Each module builds upon prior knowledge, ensuring a smooth and coherent learning experience.

Learning Objectives

Understand fundamental digital logic gates and their operations.

Design and simulate combinational logic circuits including MUX, decoders, adders, and comparators.

Implement sequential circuits including SR, JK, D, and T flip-flops.

Build sequential systems such as registers, counters, RAM, and ROM.

Gain conceptual exposure to programmable logic devices (PLDs, FPGAs).

How to Use This Manual

Each module begins with clearly stated learning objectives, followed by theoretical background, step-by-step Logisim implementation instructions, truth tables, and circuit diagrams. Exercises at the end of each section reinforce key concepts. Students are encouraged to complete all exercises before proceeding to the next module.

Prerequisites

Students should have basic familiarity with binary number systems, Boolean algebra, and fundamental logic gate operations (AND, OR, NOT). No prior experience with Logisim is required — Module 0 provides a complete installation and orientation guide.

Symbols and Notation

Symbol	Meaning
$\cdot$ (or juxtaposition)	AND operation
$+$	OR operation
$'$ (or overbar)	NOT (complement) operation
$\oplus$	XOR (exclusive-OR) operation
$\downarrow$	NOR operation
$\uparrow$	NAND operation

MODULE 0

Getting Started with Logisim

Learning Objectives

Install and launch Logisim Evolution on your computer.

Navigate the Logisim Evolution interface confidently.

Build and simulate a basic AND gate circuit.

0.1 What is Logisim?

Logisim is a free, open-source educational tool for designing and simulating digital logic circuits. It provides a graphical drag-and-drop interface, making it ideal for students learning the fundamentals of digital logic. Logisim Evolution is an actively maintained community fork that includes additional features, such as FPGA synthesis support and VHDL export.

0.2 Installation Guide

Download Logisim Evolution from the official GitHub repository:

```
https://github.com/logisim-evolution/logisim-evolution/releases
```

Choose one of the following options:

- **Installer (Recommended):** Download and run the platform-specific installer (.msi for Windows, .dmg for macOS, .deb for Linux).
- **Portable Version (.jar):** Download the .jar file and run it directly — no installation required.

Note

Java 11 or later must be installed on your computer.

To run the .jar file: double-click it, or right-click → Open with → Java Runtime.

Logisim Evolution is recommended over the original Logisim as it is actively maintained and includes built-in flip-flop components and VHDL export.

The latest stable release as of this edition is v4.0.4. Always use the latest release from the GitHub page.

0.3 Logisim Evolution Interface Overview

When you open Logisim Evolution, the interface is divided into the following main areas:

Region	Description
Menu Bar	Contains File, Edit, Project, Simulate, FPGA, Window, and Help menus. Use for saving/loading circuits, running simulations, and accessing settings.
Toolbar	Quick-access tools: Select (arrow), Wiring, Text, and Simulation controls (play, step, reset).
Component Library (Left Panel)	Lists all available components grouped by category: Wiring (pins, tunnels), Gates (AND, OR, NOT …), Memory, Arithmetic, Input/Output, etc.
Workspace (Centre)	The main grid area where circuits are designed. Click to place components; connect them with wires by dragging between terminals.
Properties Panel (Bottom-Left)	Displays and edits properties of the currently selected component (bit width, label, direction, etc.).

Figure 0.1 — Logisim Evolution interface regions and their functions

0.4 First Circuit — AND Gate

Follow the steps below to build your first circuit in Logisim Evolution. This exercise introduces component placement, wiring, and simulation.

1. Open Logisim Evolution → File → New to create a blank circuit.
2. In the Component Library (left panel), expand the Gates category.
3. Click the AND gate and place it in the centre of the workspace.
4. Expand Wiring, select Pin, and place two input pins to the left of the AND gate.
5. Place one output pin to the right of the AND gate.
6. Select the Wiring tool (wire icon in the toolbar). Click and drag to draw a wire from each input pin to an input terminal of the AND gate.
7. Wire the AND gate output to the output pin.
8. Switch to the Poke tool (hand icon). Click each input pin to toggle between 0 and 1. Observe the output pin change.

```
                    AND Gate Layout

A ──────┐
        ├──[ AND ]──── Y
B ──────┘

Boolean expression: Y = A · B
```

Figure 0.2 — AND gate circuit layout in Logisim Evolution

Expected Results

A	B	Output Y = A · B
0	0	0
0	1	0
1	0	0
1	1	1

Table 0.1 — AND Gate Truth Table

Observation	Expected Result	Verified?
Output = 0 when both inputs are LOW (0, 0)	Y = 0	☐ Pass ☐ Fail
Output = 1 ONLY when both inputs are HIGH (1, 1)	Y = 1	☐ Pass ☐ Fail
Wire changes colour to green when logic HIGH	Green wire	☐ Pass ☐ Fail

Table 0.2 — Verification Checklist (Module 0)

Exercise

Exercise 0.1: Build an OR gate using the same method. Connect two input pins and one output pin. Verify: output is 1 when at least one input is 1.

Extension: Build a NOT gate. Record its truth table (single input, inverted output).

MODULE 1

Simple Combinational Gates

Learning Objectives

Understand the operation of AND, OR, NOT, NAND, NOR, XOR, and XNOR gates.

Design combinational expressions using multiple gate types.

Implement XOR using only NAND gates (universal gate concept).

1.1 AND, OR, NOT Gates

The three fundamental logic gates are AND, OR, and NOT. All other logic functions can be derived from these three.

Gate	Symbol	Boolean Expression	Description
AND	A · B	Y = A · B	Output is 1 only when ALL inputs are 1
OR	A + B	Y = A + B	Output is 1 when ANY input is 1
NOT	A'	Y = A'	Output is the complement of the input

Table 1.1 — Fundamental Logic Gates

1.2 NAND and NOR Gates

NAND and NOR are called universal gates because any other logic function can be implemented using only NAND gates (or only NOR gates). This is significant in CMOS circuit design, where NAND gates are the preferred primitive.

A	B	AND (A·B)	NAND (A·B)'	OR (A+B)	NOR (A+B)'
0	0	0	1	0	1
0	1	0	1	1	0
1	0	0	1	1	0
1	1	1	0	1	0

Table 1.2 — AND, NAND, OR, NOR Truth Table

1.3 Practical Example — Combinational Expressions

Design and simulate the logic circuits for the following Boolean functions:

- **F = (A·B) + C'**
- **Q = (W·X) + (Y·Z)'**

Circuit F = (A·B) + C'

```
A ──┐
    ├──[AND]──────────┐
B ──┘                 ├──[OR]─── F
C ──[NOT]─────────────┘
```

Figure 1.1 — Circuit diagram for F = (A·B) + C'

Implementation Steps in Logisim Evolution:

1. Place 3 input pins and label them A, B, C.
2. Place one AND gate from Gates. Connect A and B to its inputs.
3. Place one NOT gate. Connect C to its input.
4. Place one OR gate. Connect the AND output and the NOT output to its two inputs.
5. Attach an output pin labelled F to the OR gate output.
6. Test all 8 input combinations (A, B, C = 000 to 111) and record results.

A	B	C	A·B	C'	F = (A·B)+C'
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Table 1.3 — Truth Table for F = (A·B) + C'

Circuit Q = (W·X) + (Y·Z)'

```
W ──┐
    ├──[AND1]──────────────────┐
X ──┘                          ├──[OR]─── Q
Y ──┐                          │
    ├──[AND2]──[NOT]───────────┘
Z ──┘
```

Figure 1.2 — Circuit diagram for Q = (W·X) + (Y·Z)'

Exercise

Exercise 1.1: outline the steps for designing the circuit for Q = (W·X) + (Y·Z)' in logisim evolution

Exercise 1.2: Complete the full 16-row truth table for Q = (W·X) + (Y·Z)' with all combinations of W, X, Y, Z.

Exercise 1.3: Determine the minimum SOP (Sum of Products) form for Q using a Karnaugh map.

1.4 XOR and XNOR Gates

The XOR (exclusive-OR) gate outputs 1 when its inputs differ. XNOR (exclusive-NOR) outputs 1 when the inputs are equal.

A	B	XOR (A⊕B)	XNOR (A⊕B)'
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

Table 1.4 — XOR and XNOR Truth Table

Note

XOR implementation using only NAND gates:

XOR = (A NAND (A NAND B)) NAND (B NAND (A NAND B))

This demonstrates that NAND is a universal gate — any function can be built from NAND alone.

Exercise

Exercise 1.3: Implement XOR using ONLY NAND gates. Verify using the truth table above.

Exercise 1.4: Implement F = A'B + AB' using AND/OR/NOT gates. Then confirm it equals A ⊕ B.

Extension: Implement XNOR using only NOR gates.

Common Mistakes to Avoid

Forgetting to label input/output pins — always name them clearly (A, B, Y, etc.) for simulation clarity.

Wires that appear connected but are not — always check for the filled blue dot at junctions.

Confusing NAND and NOR — NAND = NOT(AND), NOR = NOT(OR).

Mixing up XOR and XNOR — remember: XOR is HIGH when inputs DIFFER, XNOR is HIGH when inputs are EQUAL.

MODULE 2

Combinational Circuits

Learning Objectives

Design and simulate a 4:1 Multiplexer (MUX).

Implement a 3-to-8 Decoder and verify all output lines.

Build a 2-bit comparator and test all comparison cases.

Construct half and full adders and understand carry propagation.

2.1 Multiplexer (4:1 MUX)

A Multiplexer (MUX) selects one of several data input lines and routes it to a single output, controlled by select lines. A 4:1 MUX has 4 data inputs (I_0–I_3), 2 select lines (S_1, S_0), and 1 output (Y).

```
        ┌──────────────┐
I0 ─────┤              │
I1 ─────┤   4:1 MUX    ├──── Y (Output)
I2 ─────┤              │
I3 ─────┤              │
        └──────┬───────┘
               │  │
             S1  S0   (Select Lines)

Logisim: Plexers ▶ Multiplexer  (Data Bits=1, Select Bits=2)
```

Figure 2.1 — 4:1 MUX block diagram and Logisim location

S1	S0	Selected Output Y
0	0	Y = I0
0	1	Y = I1
1	0	Y = I2
1	1	Y = I3

Table 2.1 — 4:1 MUX Selection Truth Table

Logisim Implementation Steps:

1. Place 4 input pins labelled I0, I1, I2, I3; and 2 select pins labelled S0 and S1.
2. Navigate to Plexers in the Component Library. Select Multiplexer.
3. In the Properties Panel, set Data Bits = 1 and Select Bits = 2.
4. Connect I0–I3 to the MUX data inputs (top to bottom). Connect S0 and S1 to the select inputs.

5. Attach an output pin labelled Y to the MUX output.
6. Toggle S0 and S1 through all four combinations and verify the correct input routes to Y.

2.2 Decoder (3-to-8)

A decoder converts a binary code on its input lines to one active output line out of 2^n possible outputs. A 3-to-8 decoder has 3 input lines (A_2, A_1, A_0) and 8 output lines (Y_0–Y_7). Exactly one output is active (HIGH) for each unique input combination.

```
          ┌──────────────┐
A2 ───────┤              ├──── Y0  (active when A = 000)
A1 ───────┤  3-to-8      ├──── Y1  (active when A = 001)
A0 ───────┤  Decoder     ├──── Y2  (active when A = 010)
          │              ├──── Y3  (active when A = 011)
          │              ├──── Y4  (active when A = 100)
          │              ├──── Y5  (active when A = 101)
          │              ├──── Y6  (active when A = 110)
          │              ├──── Y7  (active when A = 111)
          └──────────────┘

Logisim: Plexers ▶ Decoder  (Select Bits = 3)
```

Figure 2.2 — 3-to-8 decoder with active-HIGH outputs

A2	A1	A0	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Table 2.2 — 3-to-8 Decoder Truth Table (exactly one output HIGH per row)

2.3 2-Bit Comparator

A comparator compares two binary numbers A and B and produces three mutually exclusive output signals: A > B, A = B, and A < B. Only one output is HIGH at any time.

```
A[1:0] ──┐
         ├──[COMPARATOR] ─── A > B
B[1:0] ──┘                ─── A = B
                          ─── A < B

Logisim: Arithmetic ▶ Comparator  (Bit Width = 2)
```

Figure 2.3 — 2-bit comparator: three mutually exclusive output flags

A (decimal)	B (decimal)	A > B	A = B	A < B
0 (00)	0 (00)	0	1	0
1 (01)	0 (00)	1	0	0
2 (10)	3 (11)	0	0	1
3 (11)	3 (11)	0	1	0
3 (11)	2 (10)	1	0	0

Table 2.3 — 2-Bit Comparator Sample Results

2.4 Adders

2.4a Half Adder

A half adder adds two 1-bit inputs A and B. It produces a Sum bit and a Carry bit. It cannot account for a carry-in from a previous stage — that limitation is addressed by the full adder.

```
A ──┬──[XOR]─── Sum   (Sum = A ⊕ B)
    │
B ──┴──[AND]─── Carry (Carry = A · B)
```

Figure 2.4a — Half adder using XOR and AND gates

A	B	Sum (A⊕B)	Carry (A·B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 2.4 — Half Adder Truth Table

2.4b Full Adder

A full adder adds three 1-bit inputs: A, B, and a carry-in (C_i^n). It produces a Sum and a Carry-out (C_{out}). Full adders can be cascaded to add multi-bit binary numbers.

```
  A ──┬──[XOR1]──┬──[XOR2]─── Sum
      │          │
  B ──┤      Cin ─┘
      │
  A ──┬──[AND1]──────────────┐
  B ──┘                      ├──[OR]─── Cout
  A ──┬──[AND2]──────────────┘
 Cin ─┘

  Sum  = A ⊕ B ⊕ Cin
  Cout = (A·B) + (A·Cin) + (B·Cin)
  Components: 2× XOR, 2× AND, 1× OR
```

Figure 2.4b — Full adder gate-level implementation

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 2.5 — Full Adder Truth Table

Exercise

Exercise 2A: Design a 4-bit ripple-carry adder by cascading four full adders. Connect the Cout of each stage to the Cin of the next. Test with A = 0110 (6) and B = 0101 (5). Expected: Sum = 1011 (11), Cout = 0.

Exercise 2B: Using the 4:1 MUX from Section 2.1, implement the function F(A,B,C) = Σm(1,2,5,7) by connecting appropriate logic levels (0 or 1) to the data inputs. Let A,B be the select lines.

MODULE 3

Sequential Circuits — Flip-Flops

Learning Objectives

Understand the difference between combinational and sequential logic.

Build and simulate SR, JK, D, and T flip-flops in Logisim Evolution.

Interpret state tables and timing diagrams for each flip-flop type.

Identify and explain the forbidden state of the SR flip-flop.

Sequential circuits are logic circuits whose outputs depend not only on the current inputs but also on the past history of inputs — they have memory. The fundamental storage element in a sequential circuit is the flip-flop, a bistable device that holds one bit of information.

Note

Logisim Evolution includes built-in flip-flop components (Memory ▶ D Flip-Flop, SR Flip-Flop, JK Flip-Flop, T Flip-Flop).

You can also build flip-flops from basic gates to understand their internal structure — this is the approach used in Sections 3.1 and 3.2.

A Clock component (Wiring ▶ Clock) provides the clocking signal for synchronous flip-flops.

3.1 S-R Flip-Flop (NOR-based)

The SR (Set-Reset) flip-flop is the most basic sequential element. It is built from two cross-coupled NOR gates. It has two inputs: S (Set) and R (Reset), and two complementary outputs: Q and Q'.

```
                ┌──────────────────────────────┐
                │                              │
S ──────────────▶│ NOR1 ├─── Q                 │
                │      │          ┌──────────┘
                └──────┘          │
                                  │
                ┌─────────────────┘
                │                              ┐
R ──────────────▶│ NOR2 ├─── Q'                │
                │      │          └──────────┘

Logisim: Build using 2× NOR gates with cross-coupled feedback.
```

Figure 3.1 — SR flip-flop built from two cross-coupled NOR gates

S	R	Q (Next State)	Q' (Next State)	Condition
0	0	Q (no change)	Q'	Hold / Memory
0	1	0	1	Reset
1	0	1	0	Set
1	1	Forbidden	Forbidden	Invalid — avoid this state

Table 3.1 — SR Flip-Flop State Table

⚠ Common Mistakes to Avoid

The S=1, R=1 input combination is FORBIDDEN for the NOR-based SR flip-flop.

It drives both Q and Q' to 0 simultaneously, violating the complementary output requirement.

When S and R both return to 0, the final state is unpredictable (race condition).

Circuit designers must ensure S and R are never simultaneously HIGH.

Logisim Implementation Steps:

1. Place two NOR gates from the Gates category.
2. Connect the output of NOR1 (Q) to the second input of NOR2.
3. Connect the output of NOR2 (Q') to the second input of NOR1.
4. Connect input pin S to the first input of NOR1; connect input pin R to the first input of NOR2.
5. Attach output pins labelled Q and Q' to the respective NOR gate outputs.
6. Test all valid input combinations (avoid S=R=1). Verify hold, set, and reset behaviour.

3.2 J-K Flip-Flop

The JK flip-flop eliminates the forbidden state of the SR flip-flop. When both J and K are HIGH simultaneously, the output toggles (switches to its complement). It is the most versatile flip-flop type.

```
        ┌──────────────────────┐
   J ──►│                      │
        │   J-K Flip-Flop      ├─── Q
   K ──►│                      ├─── Q'
        │                      │
 CLK ──►│                      │
        └──────────────────────┘

   Logisim: Memory ▶ JK Flip-Flop
```

Figure 3.2 — JK flip-flop block diagram

J	K	CLK Edge	Q (Next State)	Action
0	0	↑	Q (no change)	Hold
0	1	↑	0	Reset
1	0	↑	1	Set
1	1	↑	Q'	Toggle

Table 3.2 — JK Flip-Flop State Table (positive-edge triggered)

Note

The JK flip-flop is edge-triggered: state changes occur only on the rising (↑) or falling (↓) edge of the clock.

In Logisim Evolution, the built-in JK flip-flop is positive-edge triggered by default.

The Toggle mode (J=K=1) makes the JK flip-flop functionally equivalent to a T flip-flop.

3.3 D Flip-Flop

The D (Data or Delay) flip-flop has a single data input D and captures the value of D at the moment of the clock edge. It is the most commonly used flip-flop in modern digital systems, serving as the fundamental building block for registers and pipelines.

```
        ┌──────────────────────┐
  D ──▶ │                      ├─── Q
        │   D Flip-Flop        │
CLK ──▶ │                      ├─── Q'
        └──────────────────────┘

  Q(next) = D   (output simply tracks D on each clock edge)

  Logisim: Memory ▶ D Flip-Flop
```

Figure 3.3 — D flip-flop block diagram

D	CLK Edge	Q (Next State)	Action
0	↑	0	Reset — Q takes the value of D
1	↑	1	Set — Q takes the value of D

Table 3.3 — D Flip-Flop State Table

3.4 T Flip-Flop

The T (Toggle) flip-flop has a single input T. When T is HIGH at the clock edge, the output toggles; when T is LOW, the output holds its current state. T flip-flops are widely used in binary counters.

```
        ┌──────────────────────┐
   T ──►│                      ├─── Q
        │   T Flip-Flop        │
 CLK ──►│                      ├─── Q'
        └──────────────────────┘

  Q(next) = T ⊕ Q   (XOR with current state)

  Logisim: Memory ▶ T Flip-Flop  (or build from JK with J=K=T)
```

Figure 3.4 — T flip-flop block diagram

T	CLK Edge	Q (Next State)	Action
0	↑	Q (no change)	Hold
1	↑	Q'	Toggle

Table 3.4 — T Flip-Flop State Table

Flip-Flop	Inputs	Forbidden State?	Key Use Case
SR	S, R	Yes (S=R=1)	Basic latching; edge-triggered variants
JK	J, K	None	Universal; replaces SR with toggle mode
D	D	None	Registers, pipelines, data storage
T	T	None	Binary counters, frequency dividers

Table 3.5 — Flip-Flop Comparison Summary

Exercise

Exercise 3.1: Build the SR flip-flop from two NOR gates in Logisim. Verify all valid input combinations. Record what happens when you briefly set S=R=1 then return to S=R=0.

Exercise 3.2: Using the built-in D flip-flop with a Clock component, observe how Q follows D only at clock edges.

Exercise 3.3: Connect T=1 permanently to a T flip-flop and observe Q. What frequency does Q toggle at relative to the clock? What does this tell you about its use in counters?

Extension: Convert a D flip-flop to a T flip-flop using additional gates. (Hint: T = D ⊕ Q)

MODULE 4

Sequential Systems

Learning Objectives

Design and simulate a 4-bit parallel-load register.

Build a 4-bit binary ripple counter and verify its count sequence.

Implement and test RAM (Random Access Memory) in Logisim Evolution.

Implement and test ROM (Read-Only Memory) with pre-loaded data.

Gain conceptual understanding of PLDs and FPGAs.

4.1 4-Bit Register

A register is a group of flip-flops that stores multiple bits simultaneously. A 4-bit parallel-load register uses four D flip-flops sharing a common clock. On each rising clock edge, all four bits of the data input are captured and stored simultaneously.

```
D3 ──▶ [D FF] ─── Q3
D2 ──▶ [D FF] ─── Q2
D1 ──▶ [D FF] ─── Q1
D0 ──▶ [D FF] ─── Q0
         │
        CLK  (shared clock for all flip-flops)

Logisim: Memory ▶ Register  (Bit Width = 4)
```

Figure 4.1 — 4-bit parallel-load register using four D flip-flops

Logisim Implementation Steps:

1. Place four D flip-flops from Memory. Align them vertically.
2. Connect a single Clock component (Wiring ▶ Clock) to the CLK input of all four flip-flops.
3. Connect four input pins labelled D3, D2, D1, D0 to the D inputs of the respective flip-flops.
4. Connect four output pins labelled Q3, Q2, Q1, Q0 to the Q outputs.
5. Alternatively, use the built-in Memory ▶ Register component and set Bit Width = 4.
6. Load a value (e.g., 1010) on the data inputs and tick the clock. Verify Q outputs match.

Clock Cycle	D3 D2 D1 D0 (Input)	Q3 Q2 Q1 Q0 (Stored Output)
Before CLK↑	1 0 1 0	Previous value (don't care)
After CLK↑	1 0 1 0	1 0 1 0
Before CLK↑	0 1 1 1	1 0 1 0 (unchanged until next edge)
After CLK↑	0 1 1 1	0 1 1 1

Table 4.1 — 4-Bit Register Loading Behaviour

4.2 4-Bit Binary Counter

A binary counter increments (or decrements) its stored count by 1 on each clock pulse. A 4-bit binary counter counts from 0 (0000) to 15 (1111) and rolls over back to 0. It is typically built by cascading T flip-flops, each of which divides the clock frequency by two.

```
CLK ──▶ [T FF Q0] ──CLK──▶ [T FF Q1] ──CLK──▶ [T FF Q2] ──CLK──▶ [T FF Q3]
 T=1               T=1                 T=1                 T=1

All T inputs tied HIGH (T=1) so each FF toggles on every rising edge it receives.
Q0 is the LSB (toggles every cycle); Q3 is the MSB (toggles every 8 cycles).

Logisim: Memory ▶ Counter  (Bit Width = 4, Maximum Value = 15)
```

Figure 4.2 — 4-bit ripple counter using cascaded T flip-flops

Clock Pulse	Q3	Q2	Q1	Q0	Decimal Value
0 (Reset)	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	2
3	0	0	1	1	3
4	0	1	0	0	4
5	0	1	0	1	5
6	0	1	1	0	6
7	0	1	1	1	7
8	1	0	0	0	8
9	1	0	0	1	9
10	1	0	1	0	10
11	1	0	1	1	11

Clock Pulse	Q3	Q2	Q1	Q0	Decimal Value
12	1	1	0	0	12
13	1	1	0	1	13
14	1	1	1	0	14
15	1	1	1	1	15
16 (Roll-over)	0	0	0	0	0

Table 4.2 — 4-Bit Binary Counter Sequence

> **Note**
>
> Logisim's built-in Counter component (Memory ▶ Counter) is the easiest approach. Set Bit Width = 4.
>
> The ripple counter introduces propagation delay because each flip-flop's output clocks the next. For high-speed applications, a synchronous counter (all FFs share one clock) is preferred.
>
> Adding a Reset pin allows the counter to be cleared to 0 at any time.

4.3 RAM (Random Access Memory)

RAM is a volatile memory device that supports both read and write operations. Any storage location can be accessed in constant time regardless of its address — hence "random access". In Logisim Evolution, the RAM component models a synchronous byte-addressable memory array.

```
        ┌──────────────────────────┐
ADDR ──▶│                          ├─── DATA OUT
 WE  ──▶│          RAM             │
DATA ──▶│                          │
 CLK ──▶│                          │
        └──────────────────────────┘


Logisim: Memory ▶ RAM
Settings: Address Bits = 4   (16 locations)
          Data Bits = 8      (1 byte per location)
```

Figure 4.3 — RAM block diagram and Logisim settings

Logisim Implementation Steps:

1. Place a RAM component from Memory. Set Address Bits = 4 (16 locations) and Data Bits = 8.
2. Connect a 4-bit address input to the address port and an 8-bit data bus to the data port.
3. Connect a write-enable (WE) input pin: WE=1 allows writing; WE=0 selects read mode.

4. Connect a Clock component to the CLK input.
5. To write: set Address, set Data to the value you want, set WE=1, and tick the clock.
6. To read: set Address, set WE=0, and observe the data output.

Operation	WE	Address	Data In	Data Out	Result
Write	1	0000	10110101	—	Stores 0xB5 at address 0
Write	1	0001	00110011	—	Stores 0x33 at address 1
Read	0	0000	—	10110101	Reads 0xB5 from address 0
Read	0	0001	—	00110011	Reads 0x33 from address 1

Table 4.3 — RAM Read/Write Operation Examples

4.4 ROM (Read-Only Memory)

ROM is a non-volatile memory that is programmed at manufacture (or during configuration) and can only be read during normal operation. ROM is commonly used to store firmware, lookup tables, and fixed program data. In Logisim Evolution, ROM contents can be loaded from a hex file.

```
          ┌──────────────────────────┐
ADDR ──►│                          │──── DATA OUT
          │           ROM            │
          │     (Read-Only)          │
          └──────────────────────────┘


Logisim: Memory ▶ ROM
Settings: Address Bits = 4   (16 locations)
          Data Bits = 8      (1 byte per location)

Right-click the ROM ▶ Edit Contents to enter data values.
```

Figure 4.4 — ROM block diagram and content editing in Logisim

Implementation Steps:

1. Place a ROM component from Memory. Set Address Bits = 4 and Data Bits = 8.
2. Right-click the ROM component → Edit Contents.
3. Enter the following lookup table values (square of the address): 00, 01, 04, 09, 10, 19, 24, 31, 40, 51, 64, 79, 90, A9, C4, E1.
4. Close the editor. Connect a 4-bit address input pin.

5. Connect an 8-bit output pin to the data port.
6. Set the address to each value (0–15) and verify the output matches the pre-loaded data.

Address (hex)	Address (decimal)	Data Out (hex)	Data Out (decimal)	Meaning
0x0	0	0x00	0	$0^2 = 0$
0x1	1	0x01	1	$1^2 = 1$
0x2	2	0x04	4	$2^2 = 4$
0x3	3	0x09	9	$3^2 = 9$
0x4	4	0x10	16	$4^2 = 16$
0x5	5	0x19	25	$5^2 = 25$

Table 4.4 — ROM Lookup Table: Square of Address (first 6 entries shown)

4.5 Programmable Logic Devices — Conceptual Overview

Programmable Logic Devices (PLDs) are integrated circuits that can be configured by the user to implement custom digital logic. They range from simple PLAs (Programmable Logic Arrays) and PALs (Programmable Array Logic) to complex FPGAs (Field-Programmable Gate Arrays).

Device	Programmable Elements	Typical Use	Reprogrammable?
PLA (Programmable Logic Array)	AND array + OR array	Combinational logic, small functions	Often no (mask or one-time)
PAL (Programmable Array Logic)	AND array only	Registered logic, state machines	Often no
CPLD (Complex PLD)	Multiple PAL-like blocks + interconnect	Moderate-complexity control logic	Yes (flash-based)
FPGA (Field-Programmable Gate Array)	LUTs, flip-flops, DSP blocks, BRAM	Full digital systems, prototyping, ML inference	Yes (SRAM-based)

Table 4.5 — Programmable Logic Device Comparison

Logisim Evolution supports FPGA synthesis: circuits designed in Logisim can be exported as VHDL and loaded onto supported FPGA development boards (e.g., Basys 3, PYNQ). This workflow gives students hands-on exposure to the complete digital design flow from simulation to hardware.

Note

FPGA design flow: Schematic / HDL ▶ Synthesis ▶ Place & Route ▶ Bitstream Generation ▶ Download to FPGA.

HDL (Hardware Description Language) options include VHDL and Verilog. Both describe hardware behaviour at the RTL (Register-Transfer Level).

Logisim Evolution's FPGA export is a valuable first step toward learning professional FPGA toolchains such as Xilinx Vivado or Intel Quartus.

Exercise

Exercise 4.1: Build a 4-bit ring counter using 4 D flip-flops. Initialize Q = 1000. Verify that the 1 circulates: 1000 → 0100 → 0010 → 0001 → 1000.

Exercise 4.2: Modify your 4-bit binary counter to count only from 0 to 9 (BCD counter). Use a NAND gate to detect count = 10 (1010) and reset the counter asynchronously.

Exercise 4.3: Pre-load the ROM with a 7-segment display decoder truth table. Drive the ROM with a 4-bit BCD counter and connect the output to a 7-segment display component in Logisim.

Conclusion & Further Study

This laboratory manual has guided you through the fundamental building blocks of digital logic design, from basic gate-level circuits to complex sequential systems and programmable logic devices. By working through each module systematically, you have developed practical skills in:

- Designing and simulating combinational circuits (gates, MUX, decoder, adder, comparator).
- Building and analysing sequential circuits (SR, JK, D, and T flip-flops).
- Implementing sequential systems (registers, counters, RAM, ROM).
- Understanding the architecture and applications of PLDs and FPGAs.

Recommended Next Steps

To deepen your understanding of digital logic and digital systems, explore the following topics:

Topic	Description	Suggested Resources
Boolean Algebra & Karnaugh Maps	Minimise logic expressions systematically	Morris Mano — Digital Design
Synchronous State Machines	Design FSMs (Moore & Mealy models)	Roth — Fundamentals of Logic Design
Hardware Description Languages (HDL)	Learn Verilog or VHDL for professional design	Dally & Harting — Digital Design
FPGA Development	Program a real FPGA (Basys 3 / PYNQ)	Digilent Nexys / Basys documentation
Computer Architecture	Understand how logic gates form a CPU	Patterson & Hennessy — Computer Organization

Table C.1 — Recommended Further Study Topics

Assessment Checklist

Before submitting your lab reports, verify that you have completed the following:

Module	Task	Completed?
Module 0	Installed Logisim Evolution and built the AND gate circuit	☐
Module 0	Built and verified OR and NOT gate circuits	☐
Module 1	Implemented F=(A·B)+C' and verified the 8-row truth table	☐

Module	Task	Completed?
Module 1	Built XOR from NAND gates only	☐
Module 2	Built and tested 4:1 MUX, 3-to-8 Decoder, and 2-bit Comparator	☐
Module 2	Built half adder and full adder; cascaded into 4-bit ripple adder	☐
Module 3	Built SR flip-flop from NOR gates; identified forbidden state	☐
Module 3	Verified JK, D, and T flip-flop behaviour with clock	☐
Module 4	Built and tested 4-bit register and binary counter	☐
Module 4	Performed RAM read/write operations; pre-loaded and read ROM	☐

Table C.2 — Lab Completion Checklist

Digital Logic Design Lab Manual

Version 1.0 • Using Logisim Evolution

Combinational Logic • Sequential Circuits • Memory & PLDs